

# Testdoubles / Mockito Übungen

# Calculator

```
class Calculator {  
  
    /**  
     * Verwende Mockito, um ein Mock-Objekt der Calculator-Klasse zu erstellen.  
     * Simuliere das Verhalten der Methode add(int a, int b), sodass sie immer 10 zurückgibt,  
     * egal welche Argumente übergeben werden.  
     * Schreibe einen Testfall, der überprüft, ob der Mock korrekt arbeitet.  
     */  
    int add(int a, int b) {  
        return a + b;  
    }  
}
```

# UserService

```
class UserService {  
  
    private User storedUser;  
  
    /**  
     * Erstelle ein Mock-Objekt von UserService.  
     * Schreibe einen Test, der prüft, ob die Methode storeUser aufgerufen wurde,  
     * egal welche Argumente übergeben werden.  
     */  
    void storeUser(User user) {  
        if (user != null) {  
            storedUser = user;  
            System.out.println("Stored user " + user.name());  
        }  
    }  
  
    /**  
     * Erstelle ein Mock-Objekt von User und speichere ihn in ein ungemockten UserService.  
     * Schreibe einen Test, der prüft, ob der gemockte User im UserService gespeichert wurde und  
     * ob die Methode name() von dem gemockten User aufgerufen wurde.  
     */  
    User getStoredUser() {  
        return storedUser;  
    }  
}
```

```
record User(String name) {}
```

# ProductService

```
class ProductService {  
  
    /**  
     * Erstelle ein Mock-Objekt von ProductService.  
     * Simuliere verschiedene Rückgabewerte für die Methode getProductDetails(String productId):  
     * 1. Wenn die Produkt-ID „123“ ist, gebe "Product found." zurück.  
     * 2. Für alle anderen IDs gebe "Product not found." zurück.  
     * Schreibe entsprechende Testfälle.  
     */  
    String getProductDetails(String productId) {  
        return "ProductDetails";  
    }  
}
```

# PaymentService

```
class PaymentService {  
  
    /**  
     * Erstelle ein Mock-Objekt von PaymentService.  
     * Simuliere, dass die Methode processPayment() eine PaymentException auslöst, wenn der Betrag gleich 1001 ist.  
     * Schreibe einen Test, der prüft, ob die Exception korrekt geworfen wird.  
     *  
     * Bonusaufgabe:  
     * Schreibe den Test so um, dass der Mock eine Exception wirft, sobald ein Betrag größer als 1000 übergeben wird.  
     * Tipp: Schau in die Dokumentation von Mockito.  
     */  
    boolean processPayment(String creditCardNumber, double amount) throws PaymentException {  
        return true;  
    }  
}
```

```
class PaymentException extends RuntimeException {  
    PaymentException(String message) {  
        super(message);  
    }  
}
```

# OrderService

```
class OrderService {  
  
    /**  
     * Erstelle ein Mock-Objekt von OrderService.  
     * Simuliere einen Test, bei dem eine Bestellung aufgegeben wird.  
     * Prüfe, ob der placeOrder mit einem bestimmten Order-Object aufgerufen wurde.  
     */  
    public void placeOrder(Order order) {  
        // Irgendeine Bestelllogik (spielt keine Rolle)  
    }  
}
```

```
record Order(String product, int quantity) {}
```

# NotificationService

```
/**
 * Verwende Mockito, um ein Mock-Objekt von NotificationService zu erstellen.
 * Übergebe dieses Mock-Objekt über den Konstruktor von OrderService (Dependency Injection).
 * Schreibe einen Test, der die Methode placeOrder aufruft und überprüft,
 * ob die sendNotification-Methode des NotificationService mit der korrekten Nachricht aufgerufen wurde.
 */
class OrderService {

    private NotificationService notificationService;

    // Dependency Injection via Constructor
    OrderService(NotificationService notificationService) {
        this.notificationService = notificationService;
    }

    void placeOrder(String product) {
        System.out.println("Order placed for: " + product);
        notificationService.sendNotification("Order placed for " + product);
    }
}
```

```
class NotificationService {

    void sendNotification(String message) {
        System.out.println("Notification sent: " + message);
    }
}
```

# Playstore (ohne Mockito)

Gegeben sind die folgenden Klassen und folgendes Interface. Schreibe einen **Unit Test mit 100% Testabdeckung** für die Klasse **Playstore**. Schreibe hierzu einen **Mock für PlaystoreDatabase ohne Mockito**.

```
class Playstore {  
  
    private final PlaystoreDatabase database;  
  
    Playstore(PlaystoreDatabase database) {  
        this.database = database;  
    }  
  
    Collection<PlaystoreGame> searchGame(String name) {  
        try {  
            return database.findGamesByName(name);  
        } catch (IOException e) {  
            return List.of();  
        }  
    }  
}
```

```
record PlaystoreGame(String name, int rating) {  
}
```

```
interface PlaystoreDatabase {  
    List<PlaystoreGame> findGamesByName(String name) throws IOException;  
}
```



# Playstore (mit Mockito)

Schreibe einen neuen Unit Test für die Klasse Playstore (siehe vorherige Aufgabe).

Nutze dieses Mal das Framework Mockito, um die Klasse PlaystoreDatabase zu mocken.

# PlayerDamageCalculator

```
class PlayerDamageCalculator {  
  
    /**  
     * Verwende Mockito, um die statische Methode RandomUtil.randomNumber zu mocken.  
     * Schreibe einen Test für die Klasse PlayerDamageCalculator, der sicherstellt, dass die statische Methode  
     * RandomUtil.randomNumber aufgerufen wird und konstant 1 im Unit Test zurückgibt.  
     * (also keine Zufallszahl, da Zufallszahlen ungünstig als Erwartungswerte für Testfälle sind).  
     *  
     * Hinweis: Nutze mockStatic  
     *  
     * @param max: the maximum possible random number that may be generated (exclusive).  
     * @return a random number between 0 (inclusive) and max (exclusive).  
     */  
    int randomDamage(int max) {  
        return RandomUtil.randomNumber(max);  
    }  
}
```

```
import java.util.Random;  
  
class RandomUtil {  
  
    private static final int SEED = 42;  
  
    static int randomNumber(int max) {  
        return new Random(SEED).nextInt(max);  
    }  
}
```

# PlayerDamageCalculator – Dependency Injection

```
class PlayerDamageCalculator {  
  
    /**  
     * Überlege dir, wie du die Klasse PlayerDamageCalculator ohne Mockito testen kannst.  
     * Überarbeite die Klassen PlayerDamageCalculator / RandomUtil entsprechend.  
     * Eventuell benötigst du zusätzliche Klassen oder Interfaces? #hint  
     * Schreibe einen passenden Unit Test für PlayerDamageCalculator.  
     * Die Methode randomNumber soll weiterhin konstant 1 im Unit Test zurückgeben.  
     * (also keine Zufallszahl, da Zufallszahlen ungünstig als Erwartungswerte für Testfälle sind).  
     *  
     * Hinweis: Nutze KEIN mockStatic  
     *  
     * @param max: the maximum possible random number that may be generated (exclusive).  
     * @return a random number between 0 (inclusive) and max (exclusive).  
     */  
    int randomDamage(int max) {  
        return RandomUtil.randomNumber(max);  
    }  
}
```

```
import java.util.Random;  
  
class RandomUtil {  
  
    private static final int SEED = 42;  
  
    static int randomNumber(int max) {  
        return new Random(SEED).nextInt(max);  
    }  
}
```